Managing Python with Conda in Jupyter @ Hyak Klone & Tillicum

Kaichen Xie
HPC Staff Scientist

INFORMATION TECHNOLOGY
UNIVERSITY of WASHINGTON

Agenda

Introduction

UWIT Research Computing

- Services
- User support, documentation
- Student access through STF

Managing computing environment

- Modules
- Conda environment: Python
- Jupyter on Open OnDemand

Tutorial

- Objectives
- Layout
- Hands-on option

Research Computing Services

- High Performance Computing
  - Hyak- UW's Supercomputer
  - Tillicum - GPU cluster
- Data storage
  - Kopah Object Storage
  - Lolo Archive
- Research Cloud Computing
  - AWS, GCP, Azure
- Secure Computing
- Research Computing Consulting

Hyak Klone

- UW's Supercomputer
  - HPC Cluster
  - Condo Model
  - Storage
  - Community Idle Resources

- 44,184 compute cores
- 954 GPUs

Tillicum

LAUNCHED
Oct 15
2025

University of Washington's next-generation, GPU-accelerated computing platform designed to support cutting-edge research, teaching, and learning in AI, machine learning, data science, and scientific simulation.

WHERE PERFORMANCE MEETS SUSTAINABILITY

RANKED #7 FASTEST US HIGHER ED	RANKED #185 FASTEST WORLDWIDE	RANKED #51 POWER EFFICIENCY

WHY TILLICUM MATTERS?

Broad Academic Access

3	11	33
Campuses	Colleges & Schools	Departments

77	542
Research Groups	Users

Proven Demand & Impact

94,000
GPU Hours Consumed

SUPPORTING ALL DOMAINS

- Engineering
- Math
- Computer Science
- Physical & Life Sciences
- Quantum
- Teaching & Learning
- AI
- Medicine

INFORMATION TECHNOLOGY
UNIVERSITY of WASHINGTON

NVIDIA

DELL

[re]DESIGN

USER SUPPORT

INFORMATION TECHNOLOGY
UNIVERSITY of WASHINGTON

eScience Institute

Helpdesk

help@uw.edu

- Hyak, Tillicum, Kopah, Lolo, Cloud
- Research Computing Consulting
- Support Request form

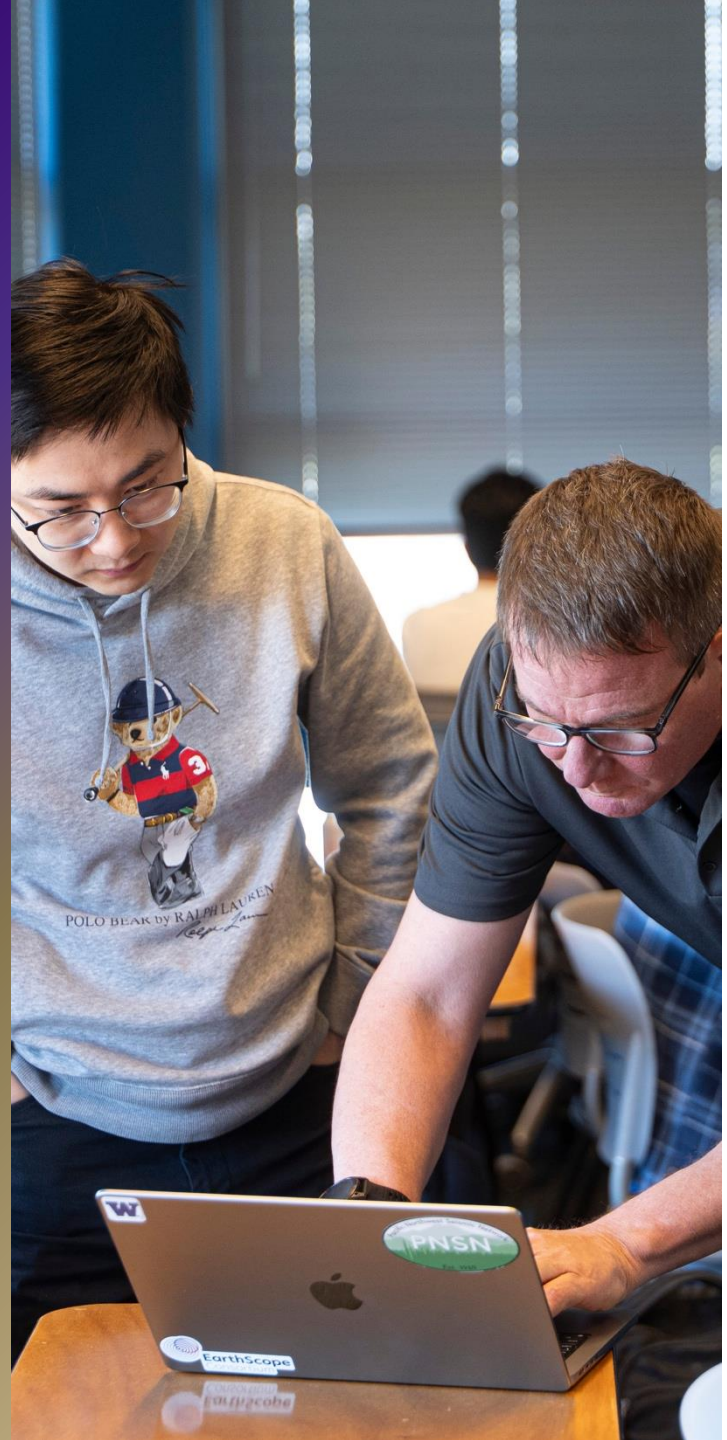
Office Hours

- Drop-in or by Appointment
  - Calendar
- In-person and remote

Regular Training Opportunities

- Recorded
- Events Calendar
- We have a mailing list

Everything underlined is a link



Documentation

www.hyak.uw.edu/

- Step by step guides to tools and software
- Monthly blog posts with updates
- Links to support and services

Learning Resources

www.hyak.uw.edu/learn

- Short How-to videos
- Full length tutorials

Everything underlined is a link

RESEARCH COMPUTING CLUB

STF | Student Technology Fee Committee

Computing Resources

Hyak Klone:

- 26 nodes
- 24 GPUs
- Priority job scheduling
- Computing storage

Tillicum:

- Coming soon: apply for $250 in Tillicum computing credits

Cloud Credits Program

- Apply for $500 in AWS credits
- Guidance and troubleshooting

Slack Community

- Get help from your peers
- Get notified about events, jobs, and fellowships

Special Events

- Hackathon
- AWS GameDay

Managing Computing Environment

INFORMATION TECHNOLOGY
UNIVERSITY *of* WASHINGTON

Main Approaches to Get Softwares

For both Hyak Klone and Tillicum Clusters:

- Loading modules
- Building containers
- Compiling from source
- Package management softwares (Conda, Spack, …)

```
 _   _                  _                 _
| | _| | ___  _ __   ___  | |__  _   _  __ _| | __
| |/ / |/ _ \| '_ \ / _ \ | '_ \| | | |/ _` | |/ /
|   <| | (_) | | | |  __/ | | | | |_| | (_| |   <
|_|\_\_|\___/|_| |_|\___| |_| |_|\__, |\__,_|_|\_\
                                  |___/

This login node is meant for interacting with the job scheduler and
transferring data to and from KLONE. Please work by requesting an
interactive session on (or submitting batch jobs to) compute nodes.

Visit the HYAK website for more documentation:
https://hyak.uw.edu/docs/

Questions? E-mail help@uw.edu with "hyak" in the subject.

Run "scontrol show res" to see any reservations in place that will
prevent your jobs from running with the reason "ReqNodeNotAvail".

Last login: Fri Feb 20 08:58:46 2026 from 98.247.40.157
[kcxie@klone-login01 ~]$
```

```
 ____ _ _ _ _                                  _   ___
|_   _(_) | (_) ___ _   _ _ __ ___     / \  |_ _|
  | | | | | |/ __| | | | '_ ` _ \   / _ \  | |
  | | | | | | (__| |_| | | | | | | | / ___ \ | |
  |_| |_|_|_|_|\___|\__,_|_| |_| |_| /_/   \_\___|

This login node is meant for interacting with the job scheduler and
transferring data to and from Tillicum. Please work by requesting an
interactive session on (or submitting batch jobs to) compute nodes.

Visit the HYAK website for more documentation:
https://hyak.uw.edu/docs/

Questions? E-mail help@uw.edu with "hyak" in the subject.

Run "scontrol show res" to see any reservations in place that will
prevent your jobs from running with the reason "ReqNodeNotAvail".

Last login: Fri Feb 20 23:20:29 2026 from 98.247.40.157
[kcxie@tillicum-login02 ~]$
```

Modules: Loading Preinstalled Software

- For Hyak Klone, module commands are disabled on login nodes.
- Example: GCC compiler

```
[kcxie@klone-login01 ~]$ gcc --version
gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-28)
[kcxie@klone-login01 ~]$ salloc -A uwit -p ckpt-all -N 1 --time=1:00:00
salloc: Pending job allocation 33446560
salloc: job 33446560 queued and waiting for resources
salloc: job 33446560 has been allocated resources
salloc: Granted job allocation 33446560
salloc: Nodes n3263 are ready for job
```

Modules: Loading Preinstalled Software

- For Hyak Klone, module commands are disabled on
- Example: GCC compiler

[kcxie@klone-login01 ~]$ gcc --version

gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-28)

[kcxie@klone-login01 ~]$ salloc -A uwit -p ckpt-all -N 1 --time=1:00:00

salloc: Pending job allocation 33446560

salloc: job 33446560 queued and waiting for resources

salloc: job 33446560 has been allocated resources

salloc: Granted job allocation 33446560

salloc: Nodes n3263 are ready for job

[kcxie@n3263 ~]$ module avail

```
------------------------------------------ /sw/klone ------------------------------------------
   abaqus/2022                      go/1.22.2                          (D)
   ansys/2020b              (D)     intel/oneAPI/2021.1.1              (D)
   ansys/2022R2                     intel/oneAPI/2023.2.1
   ansys/2023R1                     jupyter/minimal
   ansys/2024R1                     mathematica/14.1.0
   aocc/4.2.0                       matlab/r2020b
   aocl/4.2.0                       matlab/r2021a
   apptainer/local          (D)     matlab/r2023a
   apptainer/1.0.0                  matlab/r2023b                      (D)
   apptainer/1.0.1                  ngc/1.31.0
   apptainer/1.0.2                  nvhpc/22.2
   apptainer/1.0.2-1                nvhpc/24.5                         (D)
   apptainer/1.1.2                  ompi/4.1.0
   apptainer/1.1.2-1                ompi/4.1.1
   apptainer/1.1.5                  ompi/4.1.1-2
   cmake/3.20.0                     ompi/4.1.1-3
   cmake/3.21.1                     ompi/4.1.3
   cmake/3.25.1             (D)     ompi/4.1.4
   comsol/5.6                       ompi/4.1.6
   comsol/6.3               (D)     ompi/4.1.6-2                       (D)
   conda/Miniforge3-25.9.1-0        openblas/0.3.28
   cuda/11.2.2                      openjdk/23.0.1
   cuda/11.3.0                      scalapack/2.2.0
   cuda/11.4.1                      singularity/local                  (D)
   cuda/11.5.1                      singularity/1.0.0
   cuda/11.6.0                      singularity/1.0.1
   cuda/11.6.2                      singularity/1.0.2
   cuda/11.8.0                      singularity/1.0.2-1
   cuda/12.0.0                      singularity/1.1.2
   cuda/12.2.2                      singularity/1.1.2-1
   cuda/12.3.2                      singularity/1.1.5
   cuda/12.4.1              (D)     squashfuse/local                   (D)
   cuda/12.6.3                      squashfuse/0.1.104-multithread
   cuda/12.8.1                      squashfuse/0.1.105-multithread
   cuda/12.9.1                      star-ccm+/15.04.010-R8
   fftw/3.3.10                      star-ccm+/16.06.008-R8             (D)
   gcc/9.3.0                        testing/dmtcp/3.0.0
   gcc/10.2.0                       testing/matlab/parallel-server-2022b
   gcc/11.2.0               (D)     ucx/1.9.0
   gcc/12.3.0                       ucx/1.10.0
   gcc/13.2.0                       ucx/1.11.0
   gcc/15.2.0                       ucx/1.12.1
   gdrcopy/2.4.1                    ucx/1.13.1
   gnu/parallel/20210422            ucx/1.15.0
   go/1.16                          ucx/1.16.0                         (D)
   go/1.18
```

Modules: Loading Preinstalled Software

- For Hyak Klone, module commands are disabled on login nodes.
- Example: GCC compiler

```
[kcxie@klone-login01 ~]$ gcc --version
gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-28)
[kcxie@klone-login01 ~]$ salloc -A uwit -p ckpt-all -N 1 --time=1:00:00
salloc: Pending job allocation 33446560
salloc: job 33446560 queued and waiting for resources
salloc: job 33446560 has been allocated resources
salloc: Granted job allocation 33446560
salloc: Nodes n3263 are ready for job
[kcxie@n3263 ~]$ module avail
[kcxie@n3263 ~]$ module spider gcc
[kcxie@n3263 ~]$ module load gcc
[kcxie@n3263 ~]$ gcc --version
gcc (GCC) 11.2.0
[kcxie@n3263 ~]$ module load gcc/15.2.0
The following have been reloaded with a version change:
  1) gcc/11.2.0 => gcc/15.2.0
```

```
-------------------------------------------------------------------------------------------
  gcc:
-------------------------------------------------------------------------------------------
    Description:
      The GNU Compiler Collection for C, C++, Fortran, and Golang.

     Versions:
        gcc/9.3.0
        gcc/10.2.0
        gcc/11.2.0
        gcc/12.3.0
        gcc/13.2.0
        gcc/13.3.0
        gcc/15.2.0
     Other possible modules matches:
        cesg/gcc  gcc_13.3.0  ligroup/gcc  liulab/gcc

-------------------------------------------------------------------------------------------
  To find other possible module matches execute:

      $ module -r spider '.*gcc.*'

-------------------------------------------------------------------------------------------
  For detailed information about a specific "gcc" package (including how to load the modules) use th
e module's full name.
  Note that names that have a trailing (E) are extensions provided by other modules.
  For example:

     $ module spider gcc/15.2.0
-------------------------------------------------------------------------------------------
```

Modules with Lmod: Common Commands

Command	Description
`module list`	List active modules in the current environment
`module avail`	List available modules in the current environment
`module spider [module]`	Search all installed modules (deep search across all module hierarchies)
`module load [modules]`	Load modules
`module swap [module1] [module2]`	Replace `module1` with `module2`
`module unload [modules]`	Unload specific modules
`module purge`	Unload ALL modules from the current environment
`module show [module]`	Show functions performed by loading module
`module help [module]`	Show module-specific help message
`module use [-a] [path]`	Prepend or append path to `MODULEPATH`

More information:

- `man module`
- Lmod manual
- Klone documentation
- Tillicum documentation

Searching Modules

`module avail` is not for searching

- Especially for Tillicum: Tillicum has a hierarchical module system, which does not display all the modules with `module avail`.
- Only reports modules that can be loaded directly.

`module spider` will report all modules

- Performs a deep search through all module hierarchies, even ones not currently visible.
- `module spider <name>`: Report all the versions for the modules that match <name>.
- `module spider <name/version>`: Report detailed information (e.g., dependencies) on a particular module version.

```
[kcxie@tillicum-login02 ~]$ module spider cuda

-----------------------------------------------------------------------------------------------
  cuda:
-----------------------------------------------------------------------------------------------
    Description:
      NVIDIA CUDA Toolkit for GPU-accelerated computing.

     Versions:
        cuda/12.4.0
        cuda/12.9.1
        cuda/13.0.0

-----------------------------------------------------------------------------------------------
  For detailed information about a specific "cuda" package (including how to load the modules) use t
he module's full name.
  Note that names that have a trailing (E) are extensions provided by other modules.
  For example:

     $ module spider cuda/13.0.0
-----------------------------------------------------------------------------------------------
```

```
[kcxie@tillicum-login02 ~]$ module spider cuda/13.0.0

-----------------------------------------------------------------------------------------------
  cuda: cuda/13.0.0
-----------------------------------------------------------------------------------------------
    Description:
      NVIDIA CUDA Toolkit for GPU-accelerated computing.


    You will need to load all module(s) on any one of the lines below before the "cuda/13.0.0" modul
e is available to load.

      gcc/13.4.0

    Help:
      Adds CUDA Toolkit 13.0.0 to your environment.

      The NVIDIA CUDA Toolkit provides a development environment for creating high-performance, GPU-
accelerated applications. The toolkit includes GPU-accelerated libraries, debugging and optimization
 tools, a C/C++ compiler, and a runtime library.
```

Loading Modules Modifies User Environment

```
[kcxie@n3263 ~]$ module show jupyter/minimal
-------------------------------------------------------------------------------------------
   /sw/klone/jupyter/minimal.lua:
-------------------------------------------------------------------------------------------
help([[This module provides a minimal Jupyter Notebook and JupyterLab installed in a Conda environment.
Use with Open OnDemand Interactive Apps. Add custom kernels with:
  python -m ipykernel install --user --name "$env_name" --display-name "$display_name"
]])
whatis("Name: Jupyter")
whatis("Version: 2025")
whatis("Category: ood")
whatis("Description: Jupyter Notebook and JupyterLab environment via Miniforge3 for OOD")
whatis("URL: https://jupyter.org/")
prepend_path("PATH","/mmfs1/sw/miniforge3/25.9.1-0/envs/jupyter/bin")
prepend_path("PATH","/mmfs1/sw/miniforge3/scripts")
setenv("JUPYTER_CONFIG_DIR","/mmfs1/home/kcxie/.jupyter/config")
setenv("JUPYTERLAB_SETTINGS_DIR","/mmfs1/home/kcxie/.jupyter/lab")
setenv("PYTHONNOUSERSITE","1")
setenv("PIP_USER","0")
```

prepend_path: Path Changes
setenv: Environment Variables
Other Info

Using Python at Hyak: Conda Environment

INFORMATION TECHNOLOGY
UNIVERSITY of WASHINGTON

Conda Module and Custom Environment

Hyak provides a minimal conda installation that you can utilize to build custom conda environment.

- Conda allows you to create isolated and reproducible environments that include specific versions of Python, libraries, and tools.
- Load the module:

`module load conda`

Initializes conda - no need to initialize conda in you shell startup file (e.g., ~/.bashrc)

- Python:

```
[kcxie@klone-login01 ~]$ which python3
/usr/bin/python3
[kcxie@klone-login01 ~]$ python3 --version
Python 3.6.8
```

```
[kcxie@n3263 ~]$ conda activate base
(base) [kcxie@n3263 ~]$ which python
/mmfs1/sw/miniforge3/25.9.1-0/bin/python
(base) [kcxie@n3263 ~]$ python --version
Python 3.12.12
```

✗ System default - not the Python we want!

✓ Always use Python from a conda environment

Create a Custom Conda Environment

1. Customize environment and package locations:

- The default location for custom conda environment is your home directory, which has a storage quota of only 10 GB.
- Option 1: Use --prefix for explicit paths

```
$ module load conda
$ conda create --prefix /gscratch/<myproject>/<myfolder>/myenv python=3.12
$ conda activate /gscratch/<myproject>/<myfolder>/myenv
```

- Option 2: Configure defaults in $HOME/.condarc (recommended)

```
$ nano ~/.condarc
envs_dirs:
 - /gscratch/<myproject>/<myfolder>/conda/envs
pkgs_dirs:
 - /gscratch/<myproject>/<myfolder>/conda/pkgs
always_copy: true
$ module load conda
$ conda create -n myenv python=3.12 numpy scipy pandas matplotlib
$ conda activate myenv
```

Package installation

`conda install`:

- Packages installed via conda can come from different *channels*
- In many cases it's fine to mix packages from different channels and/or pip but this can sometimes lead to version conflicts.
- Check the packages installed in your environment with `conda list`

`pip install`:

- Best practice: use pip inside a conda environment to install Python packages into the current environment

```
$ module load conda
$ conda activate myenv
$ pip install torch torchvision
```

- Otherwise, pip installs packages into a local cache. On Hyak, pip will automatically fall back to a --user install (in $HOME/.local/lib/python<version>)

Using GPUs with Python

cuda module:

- Any cuda-aware packages should be built on a GPU node.
- Some GPU-enabled packages installed from conda-forge will install cudatoolkit into your conda environment. This may conflict with the cudatoolkit module that is loaded.

cudatoolkit dependency via conda/pip:

- cupy from conda-forge channel will pull cudatoolkit dependencies into conda env:

```
$ salloc -A uwit -p ckpt-all --gpus-per-node=2080ti:1 --time=1:00:00
$ module purge
$ module load conda
$ conda create --name cupy-test python=3.11 numpy
$ conda activate cupy-test
$ conda install -c conda-forge cupy
$ python
>>> import cupy as cp
```

Using Jupyter at Hyak: Open OnDemand

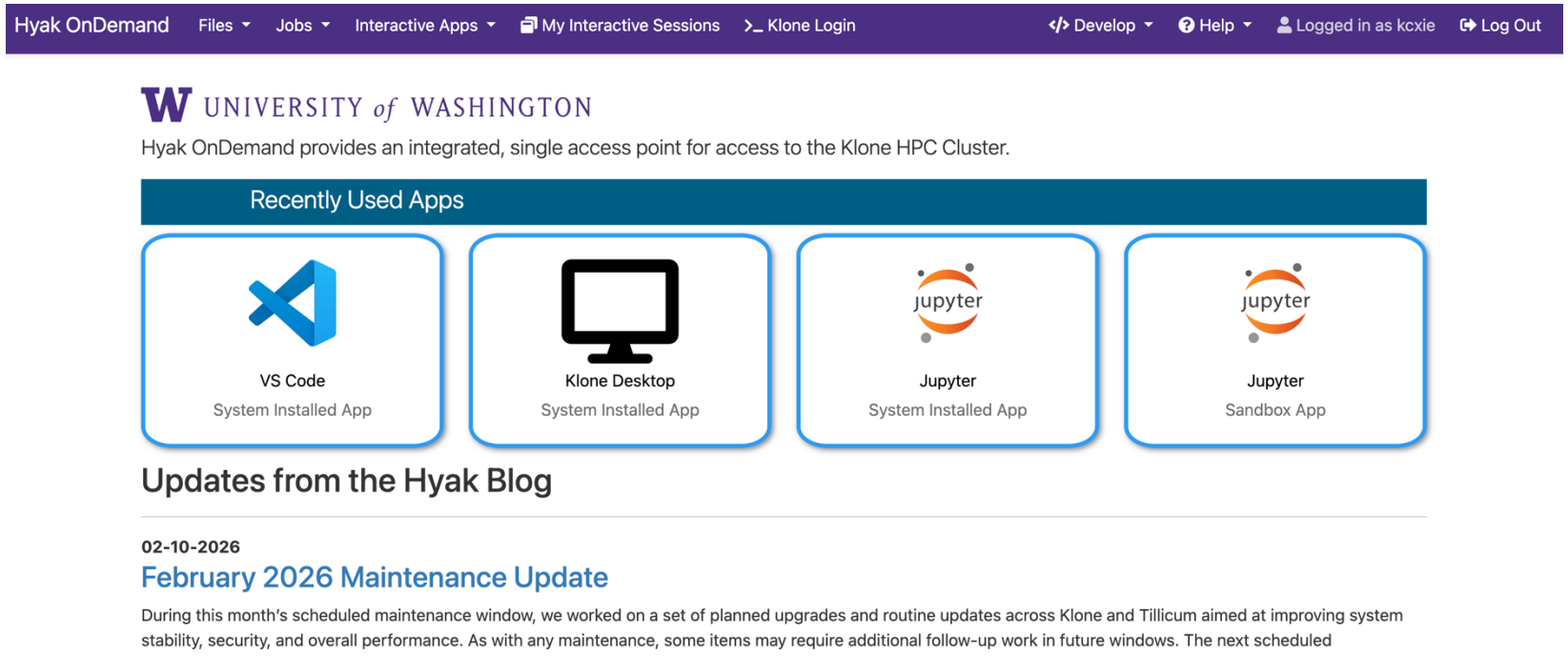
INFORMATION TECHNOLOGY
UNIVERSITY of WASHINGTON

How Do I Use Jupyter at Hyak: Open OnDemand

Hyak Klone: https://ondemand.hyak.uw.edu/

Tillicum: https://tillicum-ood.hyak.uw.edu/

1. Authenticate: If you access OOD from off campus, you will need to connect to the UW VPN. You can find instructions on how to connect to the UW VPN here.

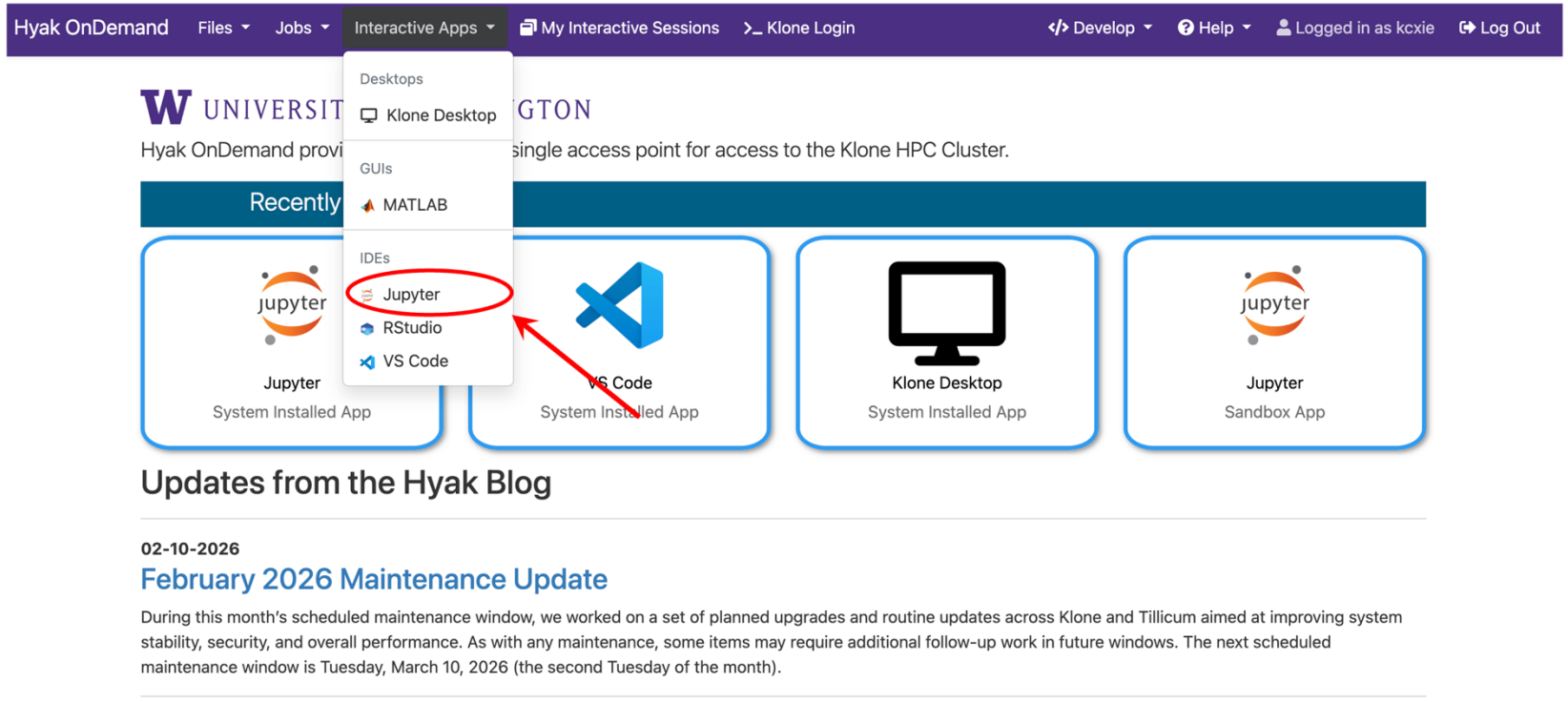


How Do I Use Jupyter at Hyak: Open OnDemand

Hyak Klone: https://ondemand.hyak.uw.edu/

Tillicum: https://tillicum-ood.hyak.uw.edu/

2. Navigate to the Jupyter App

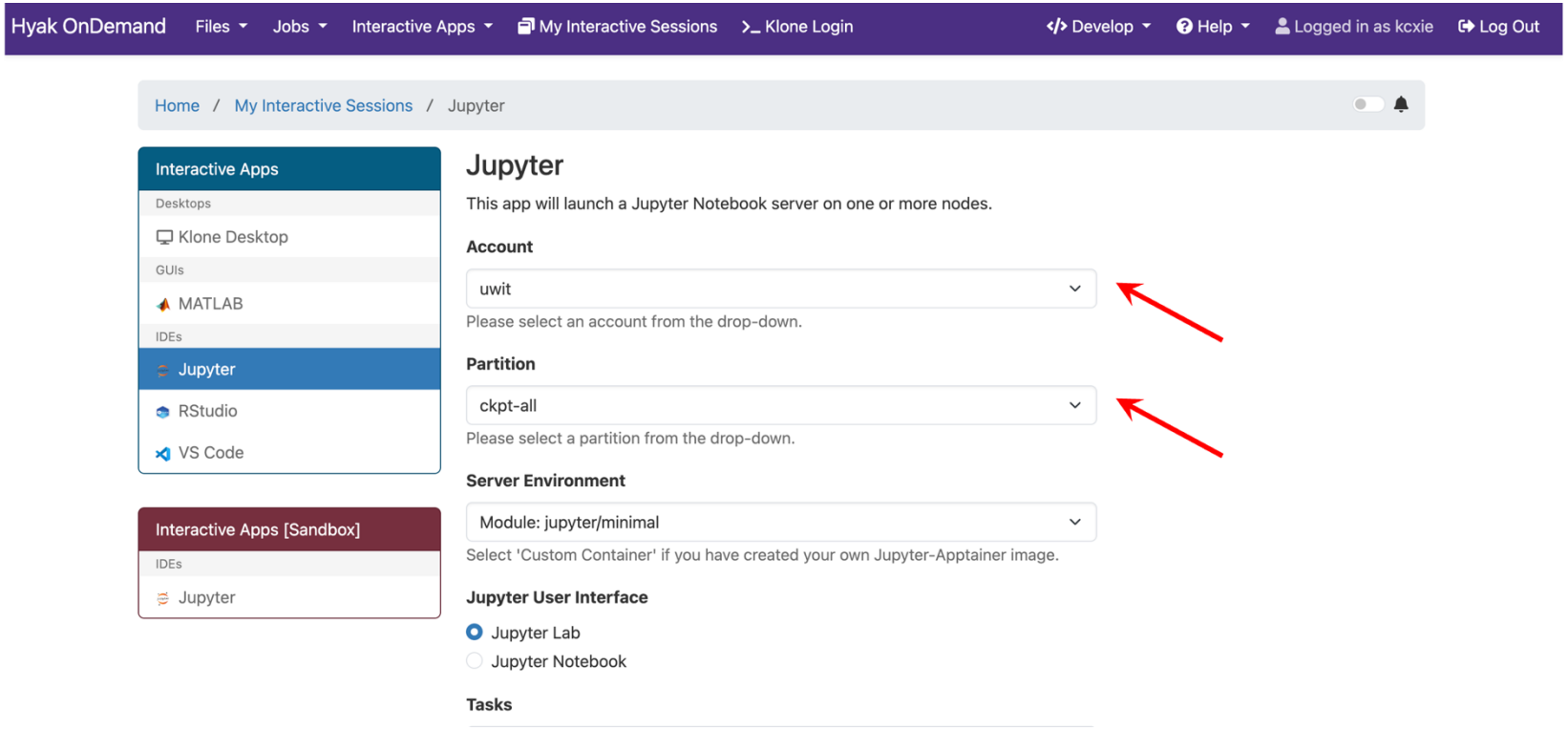


How Do I Use Jupyter at Hyak: Open OnDemand

Hyak Klone: https://ondemand.hyak.uw.edu/

Tillicum: https://tillicum-ood.hyak.uw.edu/

3. Configure the interactive job and launch the job

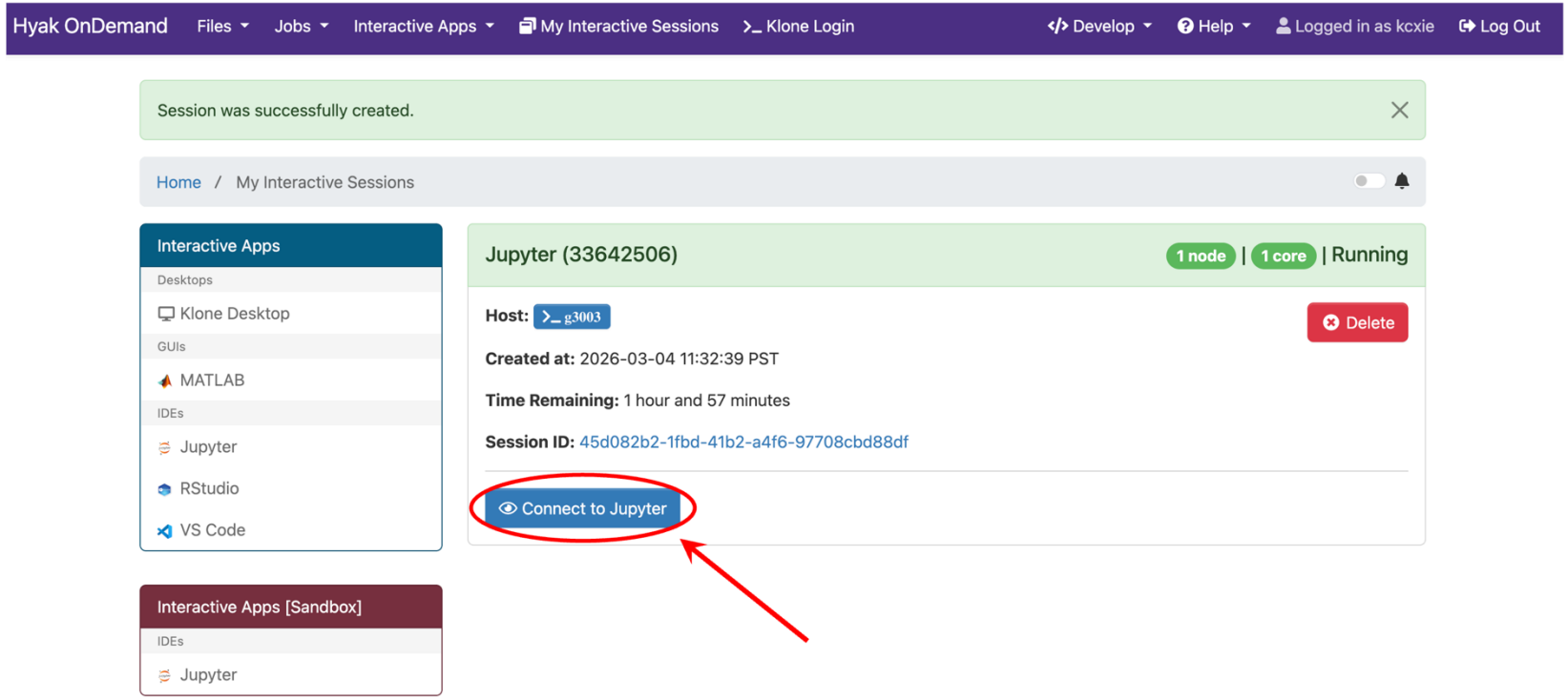


How Do I Use Jupyter at Hyak: Open OnDemand

Hyak Klone: https://ondemand.hyak.uw.edu/

Tillicum: https://tillicum-ood.hyak.uw.edu/

4. Connect to Jupyter

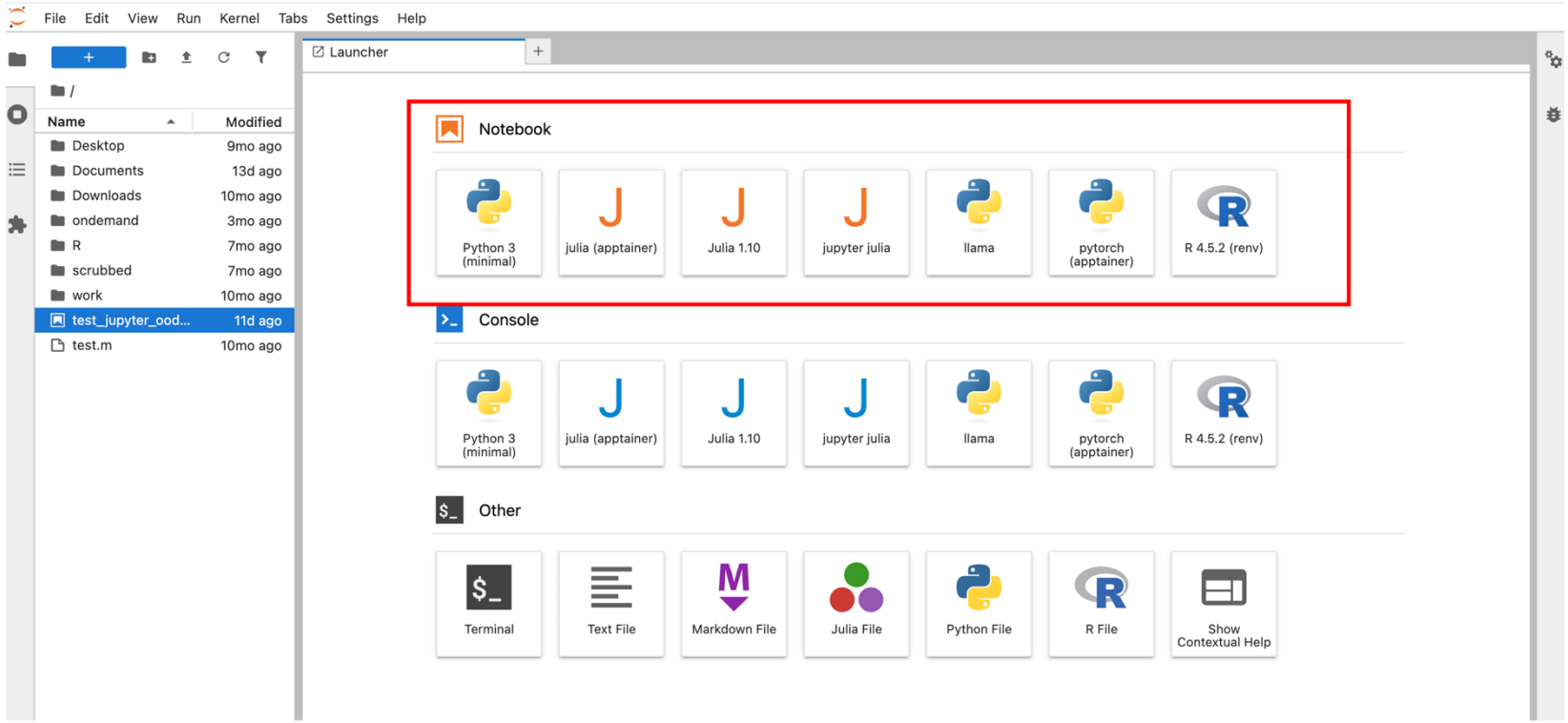


How Do I Use Jupyter at Hyak: Open OnDemand

Hyak Klone: https://ondemand.hyak.uw.edu/

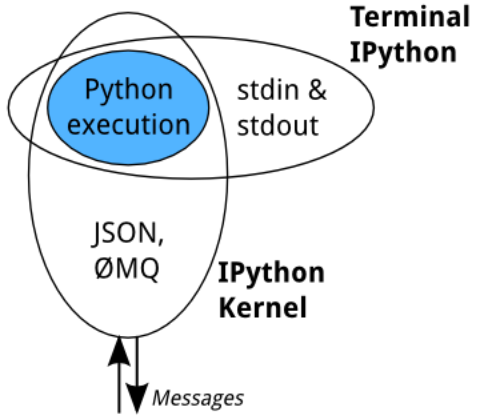
Tillicum: https://tillicum-ood.hyak.uw.edu/

5. Start a new notebook with a Jupyter kernel

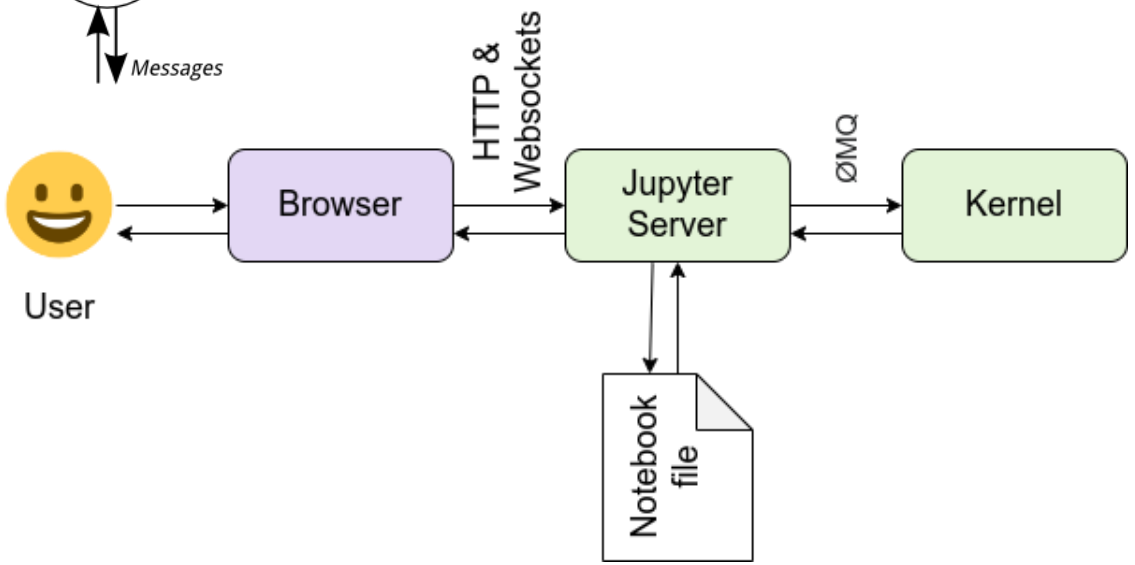


Jupyter Kernels

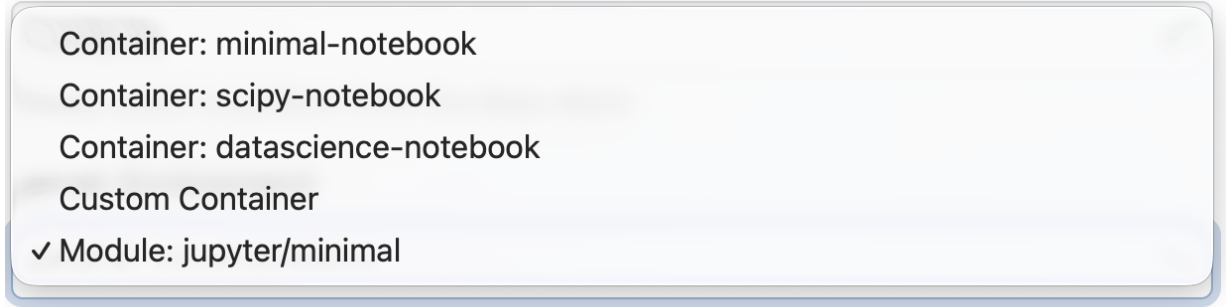
- The IPython Kernel is what actually runs your code.



- Jupyter Notebook: run and store your code, output, markdown notes



- Server environment options for Jupyter on OOD
- Default kernel for Module: jupyter/minimal is Python3 (minimal)



Select 'Custom Container' if you have created your own Jupyter-Apptainer image.

Custom Jupyter Kernel: Conda Environment

How to use a conda environment as a IPython Kernel?

- Several ways to accomplish this; we recommend:

```
$ module load conda
$ conda create -n myenv python=3.11
$ conda activate myenv
(myenv) $ conda install ipykernel <other packages>
(myenv) $ python -m ipykernel install --user --name myenv --display-name "Python (myenv)”
```

-> This creates a “kernelspec” file: $HOME/.local/share/jupyter/kernels/myenv/kernel.json

- Switch back to your JupyterLab interactive session
- Kernel “Python (myenv)” should be present in the kernel list

The kernelspec File

```
$ cat $HOME/.local/share/jupyter/kernels/myenv/kernel.json
{
 "argv": [
  "/mmfs1/gscratch/scrubbed/<user>/conda/envs/myenv/bin/python",
  "-m",
  "ipykernel_launcher",
  "-f",
  "{connection_file}"
 ],
 "display_name": ”Python (myenv)",
 "language": "python",
 "metadata": {
  "debugger": true
 }
}
```

More customization:
See kernel specs in Jupyter documentation.

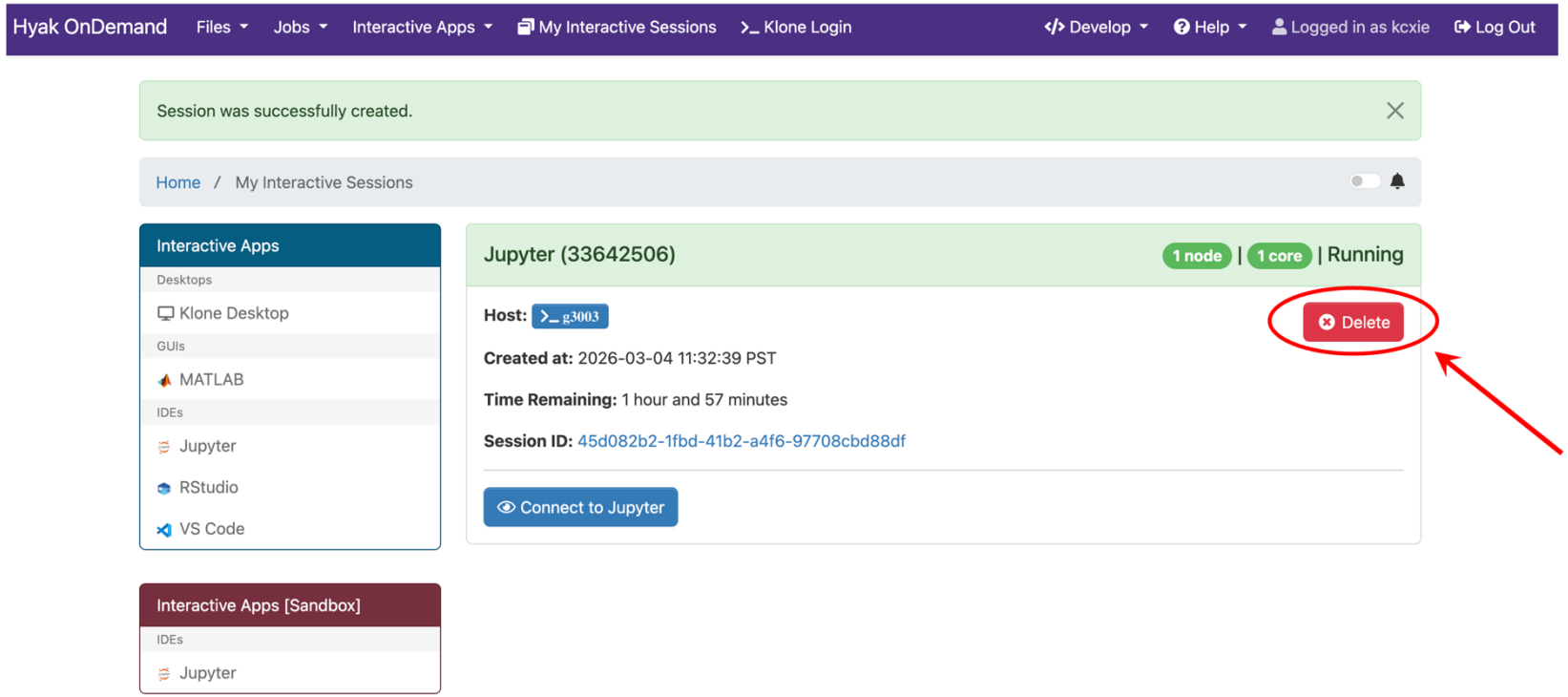
An Apptainer kernelspec File

```
$ cat $HOME/.local/share/jupyter/kernels/pytorch/kernel.json
{
 "argv": [
  "apptainer",
  "exec",
  "--cleanenv",
  "--bind", "/gscratch",
  "</path/to/your/container/image>/<image_name>.sif",
  "</path/to/your/image/python>",
  "-m",
  "ipykernel_launcher",
  "-f",
  "{connection_file}"
 ],
 "display_name": "Python (My Environment)",
 "language": "python",
 "metadata": {
  "debugger": true
 }
}
```

Close the Jupyter Session

When you finish:

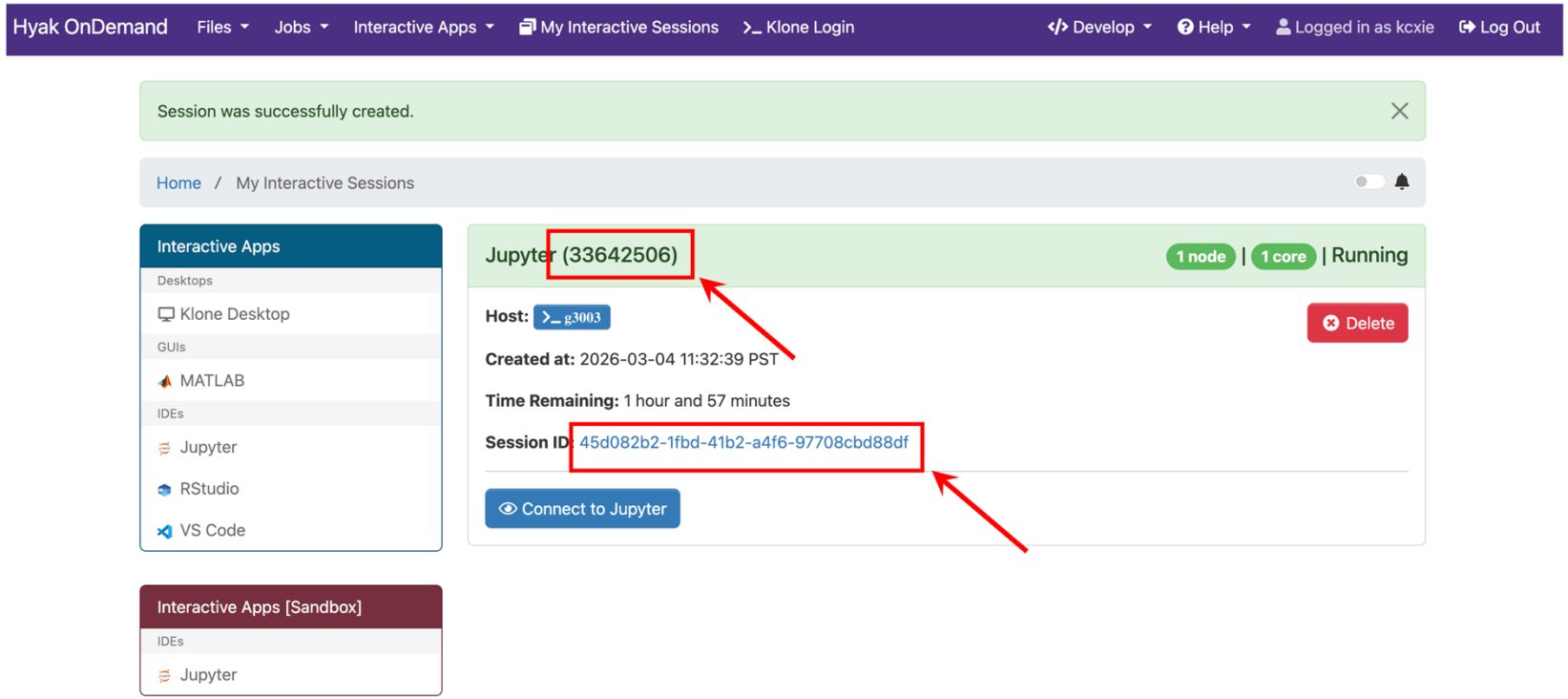
- Save your work and close the Jupyter browser tab.
- Return to OOD My Interactive Sessions and click Delete on your running session card.
- For Tillicum, leaving sessions running consumes GPU hours and counts toward your project usage.



Debugging Jupyter Issues

If you run into any issues, reach out to help@uw.edu with:

- Slurm Job ID — the scheduler identifier (the number in the parentheses).
- Session ID — links to detailed logs for troubleshooting.



Learning Objectives

By completing this tutorial, you'll learn how to:

- Manage computing environment with modules and conda environment
- Create your own conda environment and install packages
- Launch Jupyter server on Hyak Klone using Open OnDemand
- Register conda environment as Jupyter kernel

Training Materials Repo:

https://github.com/UWrc/jupyter-tutorial.git

Feedback Survey:

INFORMATION TECHNOLOGY
UNIVERSITY of WASHINGTON

Learning Objectives

INFORMATION TECHNOLOGY
UNIVERSITY of WASHINGTON

NEED TO KNOW'S: training access period

- You will be able to complete whenever you want with your demonstration account (or other)
- Over the next ~3 days (ending Monday morning 3/9) you will have priority access via UWIT (slurm account "uwit")
- This tutorial is designed as introductory concepts and practice followed by a "task" that combines multiple skills.

Training Materials Repo:

https://github.com/UWrc/jupyter-tutorial.git

Feedback Survey:

Learn more – everything underlined is a link

UWIT - https://it.uw.edu/

Research Computing Services
- Tillicum: GPU-Accelerated Research Computing Platform
  - Intake Form
- Hyak: High-Performance Supercomputing Research Cluster
  - Pricing and Eligibility
  - Documentation
- Data Storage Services
  - Kopah S3 Object Storage
    - Pricing and Eligibility (cost calculator)
    - Documentation
  - Lolo Data Archive
    - Pricing and Eligibility
    - Documentation
- Cloud Computing
- Computing for Restricted Access Data
- Research Computing Consulting

Training and Events
- Hyak mailing list
- Hyak Blog
- UWIT Research Computing Calendar
- Office Hours
- eScience Newsletter (scroll for sign up)
- eScience Data Science and AI Accelerator
- Office of Research Calendar

Past Trainings
- Research Computing Tutorials & Video Library
- eScience YouTube Channel

New Hyak Users
- Free Demonstration Account
- Hyak Basics Tutorial
- Limitations of demonstration accounts
- Using free resources (Checkpoint)

Students
- Research Computing Club
- Student Hyak account
- Cloud Credits
- Student Technology Fee

Other Resources
- **Join the UW AI Community of Practice** on MS Teams to get updates from UW-IT's AI team about events and join the discussion around AI in the news, society, and culture.
- UW Seattle Events Calendar

Special thanks to
- UW Office of Research
- eScience Institute